WattWise

Compiler-Assisted Energy Profiling and Optimization

Final Report: Demonstration, Evaluation and Project Completion Review

Item	Details
Course Code / Title	BCSE307 – Compiler Design
Team Number	A35
Project ID and Title	Project ID – WattWise: Compiler-Assisted Energy Profiling and Optimization
Review Date	dd Month yyyy
Faculty / Evaluator	Faculty name
Source repository	https://github.com/abhithapliyal05/WattWise

Team members

Name	Register number
Abhyuday Thapliyal	24BCT0204
Arhat Gupta	24BDS0195
Anaya Singh	24BCI0077
Keshav Biyani	24BCE0344

Originality declaration

We declare that this report and the accompanying software are the work of the team named above, with the third-party code, libraries, tools and assistance acknowledged in the Acknowledgements section and in the references. We have not included confidential credentials, personal data or unapproved copyrighted material.

Team leader signature: _____ Date: _____

Contents

Acknowledgements and use of third-party resources **3**

Abstract **4**

1. Introduction and Motivation **5**

2. Problem Statement, Objectives, Scope and Limitations **5**

2.1 Problem statement 5
2.2 Objectives and compliance 5
2.3 Scope 6
2.4 Limitations in brief 6

3. Background and Related Work **6**

3.1 Software energy models 6
3.2 Hardware energy counters 6
3.3 Compiler analysis and optimisation 6
3.4 Static frequency estimation and profile guidance 6
3.5 Differential testing 7
3.6 Positioning 7

4. Requirements and Technology Stack **7**

5. System Design **8**

5.1 Architecture and data flow 8
5.2 Modules and interfaces 8
5.3 Core data structures 9
5.4 Design decisions 9
5.5 Command-line interface 9

6. Algorithms and Methodology **10**

6.1 Lexical analysis 10
6.2 Parsing 10
6.3 Semantic analysis 10
6.4 Intermediate code 10
6.5 Control-flow graph, dominators, loops and liveness 10
6.6 Energy model 11
6.7 Execution frequency 11
6.8 Aggregation and the hot set 12
6.9 Optimisation passes 12
6.10 Verification 13

7. Implementation Details **14**

7.1 Code organisation 14
7.2 Build process 14
7.3 The interpreter 14
7.4 Continuous integration 14
7.5 Web interface 14

8. Testing Strategy and Test Cases **16**

8.1 Layers of testing 16
8.2 Representative test cases 17
8.3 Boundary, failure and stress cases 17
8.4 Regression and what the tests caught 17

9. Results and Evaluation **18**

9.1 Correctness 18
9.2 Optimisation effectiveness 18
9.3 What the Review 3 passes add 19
9.4 Static estimate against measured counts 20
9.5 Cost of the analysis 21
9.6 Threats to validity 21

10. Error Handling, Limitations and Known Issues **22**

10.1 Error handling 22
10.2 Limitations and known issues 22

11. Member-wise Contribution **22**

12. Conclusion and Future Work **23**
12.1 What was achieved 23
12.2 What we learned 23
12.3 Future work 23
13. References **24**
Appendix A. User Manual **25**
A.1 Build and run 25
A.2 Web interface 25
A.3 Tests 25
Appendix B. Sample Runs **25**
B.1 IR and control-flow graph of hoist.cpp 25
B.2 Optimised IR of hoist.cpp 26
B.3 Text report for hoist.cpp with --opt 26
B.4 Optimisation log for redundant.cpp 27
Appendix C. Complete Test Table **28**
C.1 Invalid programs 28
C.2 Differential suite 29
C.3 Unit test cases 29
Appendix D. Grammar of the Accepted Subset **30**
Appendix E. Intermediate Representation **30**
Appendix F. Contribution Log (to be completed) **31**
Appendix G. Evaluator-Selected Test Record **31**

Acknowledgements and use of third-party resources

We thank faculty name for guidance throughout the course project. The following external resources are used and acknowledged:

- **Catch2** v3.5.4 (Boost Software License), vendored in `third_party/` and used only by the unit tests.
- **GCC / g++** (used as the independent correctness oracle and as the build compiler), **CMake**, **Ninja**, **Node.js** and **Express** (web interface), and **GitHub Actions** (continuous integration).
- The instruction-level energy-modelling literature (Tiwari et al.; Steinke et al.), from which the relative ordering of the cost table is taken. The constants themselves are the team's own and are not copied from those papers.
- The Dragon Book and Muchnick's textbook for the textbook formulations of dominators, natural loops, liveness and the classical optimisations.

AI assistance

Confirm this paragraph against your course's policy on AI assistance before submitting. Parts of this project were developed with the assistance of an AI coding assistant (Anthropic's Claude, used through Claude Code). It was used for: the redundant-load and dead-code elimination passes and their unit tests; extending common-subexpression elimination to extended basic blocks; the `redundant.cpp` benchmark; the web interface (server and browser front end); a continuous-integration fix; documentation updates; and drafting this report. Every number in this report was produced by running the code in this repository, and the team remains responsible for the content.

Abstract

Energy use has become a design constraint for software, yet most programmers cannot see which parts of a program are expensive without running it on instrumented hardware. WattWise is a small compiler for a restricted subset of C++17 that estimates where a program spends energy and rewrites the hottest code to spend less. It lowers source code to three-address code, builds a control-flow graph with dominators, natural loops and liveness, and prices every instruction with a documented, overridable table of per-operation costs in model picojoules. Execution frequencies come either from a static estimator based on loop induction variables and the call graph, or from an IR interpreter that counts real executions; the tool compares the two rankings. Energy is aggregated per block, loop, function, source line and operation class, and the lines covering 90% of the total form the hot set. Five profile-directed passes (strength reduction, common-subexpression elimination over extended basic blocks, loop-invariant code motion, redundant-load elimination and dead-code elimination) rewrite only that hot code, and the interpreter then checks that the program's output is unchanged. Every accepted program is also valid C++, so g++ serves as an independent oracle. Across 15 benchmarks the passes reduce estimated energy by 10.7% on average (12.8% under measured execution counts) and by up to 58.6%. All 35 differential tests, 33 invalid-program tests and 58 unit tests pass, and continuous integration is green. The static estimator ranks hotspots well (the top block is correct on 14 of 15 programs) but misjudges magnitude when loop bounds are unknown, and the cost model is relative rather than calibrated to hardware.

1. Introduction and Motivation

Software runs on batteries and in data centres, and the energy it uses is shaped by decisions made long before the program executes: which algorithm is chosen, how loops are written, and which instructions the compiler finally emits. Compilers already transform programs to make them faster or smaller. Energy-aware compilation applies the same machinery with a different objective: find the code that costs the most energy and make it cheaper without changing what the program computes.

The practical difficulty is visibility. A programmer who asks "where does this program spend its energy, and what would change that?" has two poor options. Measuring real hardware needs instrumented machines, reports totals over a time window rather than per source line, and is hard to repeat. Instruction-level energy models attribute cost per instruction, but a model on its own does not tell a programmer which lines to change, and a model with no way to apply and check the changes stops short of being useful.

WattWise closes that loop in a form small enough to read end to end. It accepts a subset of C++, compiles it to an intermediate representation, estimates the energy of every basic block, loop, function and source line, ranks the hotspots, rewrites the hot code with classical optimisations, re-estimates, and verifies by execution that the optimised program prints the same output. Because the accepted language is real C++, every test program can also be compiled with g++ and compared, which turns the compiler into its own best test subject.

The intended beneficiaries are students and instructors, for whom every classical compiler phase is present and inspectable in about four thousand lines of C++, and developers who want a fast, repeatable way to compare how code variants rank in energy before measuring on a device. The project touches most of the course: lexical analysis, parsing with error recovery, semantic analysis, intermediate code generation, control-flow and data-flow analysis, and code optimisation, together with the engineering needed to test a compiler convincingly.

2. Problem Statement, Objectives, Scope and Limitations

2.1 Problem statement

Given a program in a restricted subset of C++, estimate how much energy each part of it uses under a documented cost model, identify the parts that matter, transform them to use less, and demonstrate that the transformation preserves the program's observable behaviour.

2.2 Objectives and compliance

Drafting note, delete before submission: the objectives below were reconstructed from the repository README and docs. Replace or reconcile them with the objective list approved at Reviews 1 and 2.

Table 1. Objective-compliance matrix.

ID	Objective	Status	Evidence
O1	Accept a small subset of C++ that is itself valid C++, so g++ can act as an oracle.	Met	Subset grammar in Appendix D. All 35 valid programs match g++ (Section 9.1).
O2	A complete front end with error recovery and located diagnostics.	Met	33 of 33 invalid programs rejected in the expected phase (Appendix C).
O3	Three-address IR, control-flow graph, dominators, natural loops and liveness.	Met	8 IR and CFG unit tests; dumps in Appendix B.
O4	A documented, overridable instruction-level cost model.	Met	Table 7 and the `--costs FILE` option.
O5	Static and dynamic execution frequencies, with a cross-check between them.	Met	Top block agrees on 14 of 15 programs (Section 9.4).
O6	Energy ranking by block, loop, function, source line and operation class.	Met	Text, JSON and Graphviz reports; web interface.

ID	Objective	Status	Evidence
O7	Energy-directed optimisation passes.	Met	26 rewrites over 15 benchmarks; mean static reduction 10.7%.
O8	Optimisation must never change program output.	Met	Verification oracle in the CLI; 35 of 35 differential tests.
O9	A usable interface for demonstration.	Met	Command line and browser interface (Section 7.5).
O10	Calibrate the cost table against hardware energy counters.	Not attempted	Future work (Section 12).

2.3 Scope

In scope: `int`, `double`, `bool` and `void`; one-dimensional arrays; global and local scalars; functions with scalar parameters, including recursion; `if`, `while`, `for`; compound assignment and increment as statements; `cout` output. **Out of scope:** classes, templates, pointers and references, multi-dimensional arrays, array parameters, `switch`, `break` and `continue`, input, threads, the standard library containers and floating-point formats other than `double`. These exclusions are deliberate: they remove aliasing and side effects inside expressions, which keeps every optimisation in this report easy to state and to prove safe.

2.4 Limitations in brief

The cost model is a relative table, not a hardware measurement. The static frequency estimator is exact only for loops with constant start, step and bound. The optimiser has five passes and does not forward values across loop iterations. Section 10 gives the full list and Section 12 the plan for addressing it.

3. Background and Related Work

3.1 Software energy models

Tiwari, Malik and Wolfe [1] measured the current drawn by individual instructions and by pairs of instructions on real processors and built an instruction-level model from those measurements, with a base cost per instruction plus an inter-instruction overhead. Steinke et al. [2] refined this style of model and used it to guide software optimisations. These works establish the idea WattWise relies on: the energy of a program can be estimated as the sum over executed instructions of a per-instruction cost, and the useful property of such a table is its ordering (memory accesses and divisions cost far more than additions and moves).

3.2 Hardware energy counters

Modern processors expose energy estimates through interfaces such as Intel RAPL [3], which report energy per package, core or memory domain over a time window. They are well suited to measuring whole-run totals on a real machine. They do not attribute energy to individual instructions or source lines, and on some processors the values are themselves model-based estimates. WattWise does not read them; Section 12 describes how they could be used to calibrate its cost table.

3.3 Compiler analysis and optimisation

Dominators, natural loops, liveness, loop-invariant code motion, common-subexpression elimination, strength reduction and dead-code elimination are standard material [4], [5]. WattWise implements the textbook formulations and adds the policy layer that is specific to energy: the passes act only on the code that the energy analysis marks hot.

3.4 Static frequency estimation and profile guidance

Optimisers need to know how often code runs. Profile-guided optimisation measures this with instrumented runs. Static alternatives predict branch outcomes and loop counts from program structure [6], [7]. WattWise

supports both: a deliberately simple static estimator, and exact counts from its own interpreter, so that the two can be compared on every program.

3.5 Differential testing

Differential testing runs one input through independent implementations and flags any disagreement [8]. Because every WattWise-accepted program is valid C++, g++ is an independent implementation of the same language, and the project tests itself against it.

3.6 Positioning

Table 2. How the approaches compare. The WattWise row describes this project, not a measured comparison with the others.

Approach	Needs hardware	Attributes energy to source lines	Rewrites the code	Checks the rewrite
Hardware counters (RAPL, power meters)	Yes	No (time windows)	No	No
Instruction-level model [1], [2]	For calibration	Possible in principle	Not by itself	Not by itself
Profile-guided optimisation	No	No (optimises for speed)	Yes	By the compiler's own testing
WattWise	No	Yes, from a relative model	Yes, hot code only	Yes, by execution against g++ and the interpreter

We do not claim a new energy model. The contribution is integration and inspectability: one small, readable toolchain in which the cost model, the frequency estimate, the ranking, the rewriting and the verification are all visible and testable.

4. Requirements and Technology Stack

Table 3. Software, hardware and data used.

Item	Choice	Notes
Implementation language	C++17	No third-party runtime dependencies in the compiler.
Compilers	g++ 13.2.0 (MSYS2 UCRT64) and GCC 13.3.0	Development on Windows 11; CI on Ubuntu 24.04. The README states g++ 11 or clang 14 as the minimum.
Build system	CMake 3.16 or newer, Ninja	CMake 4.3.0 used. Warnings: `-Wall -Wextra -Wpedantic`.
Unit tests	Catch2 3.5.4 (amalgamated)	Vendored in `third_party/`.
Other tests	Bash scripts, g++ as oracle	`scripts/difftest.sh`, `scripts/invalidtest.sh`.
Continuous integration	GitHub Actions	Configure, build and `ctest` on every push.
Web interface	Node.js 18+ (v24.14.1 used), Express 4.22.3	Plain HTML, CSS and JavaScript; no framework and no CDN.
Test programs	15 benchmarks, 20 valid programs, 33 invalid programs	All in the repository (`benchmarks/`, `tests/`).
Repository	https://github.com/abhithapliyal05/WattWise	Branch `main`.

5. System Design

5.1 Architecture and data flow

WattWise is a pipeline. The front end (M1) turns text into a typed syntax tree. The middle end (M2) lowers it to three-address code and builds the control-flow graph. The analysis stage (M3) combines a frequency source with the cost model to produce an energy report and a hot set. The optimiser rewrites the hot code, and the verifier re-analyses the result and runs both versions to confirm the output is unchanged. All of this lives in one static library, `wattwise_core`; the command-line tool, the unit tests and the web server all use the same code.

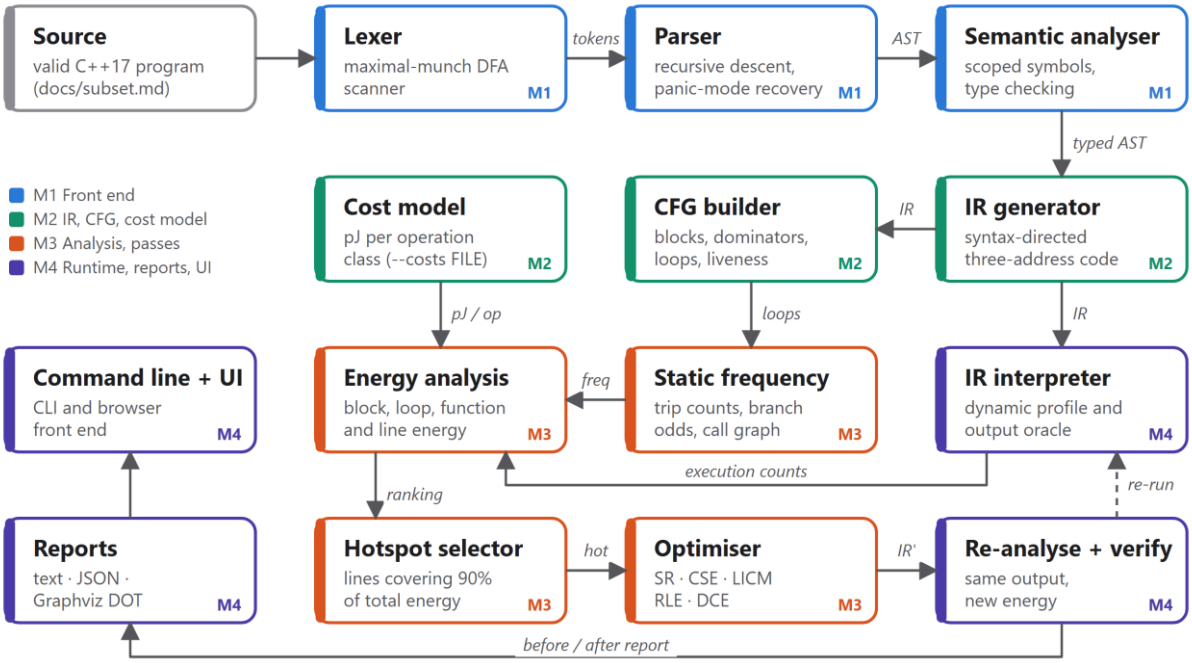


Figure 1. WattWise pipeline. Colours mark the module groups from the project README (M1 to M4).

5.2 Modules and interfaces

Table 4. Modules, files and interfaces. Line counts include headers.

Module	Files	Input → output	Group	Lines
Lexer	`lexer.cpp`	Source text → tokens	M1	264
Parser	`parser.cpp`, `ast.hpp`	Tokens → AST	M1	559
Semantic analysis	`semantic.cpp`	AST → annotated AST, diagnostics	M1	485
Common types	`common.cpp/.hpp`	Diagnostics, `CompileError`	M1	65
IR and generator	`ir.cpp`, `irgen.cpp`	Annotated AST → quadruples	M2	655
CFG	`cfg.cpp`	Quadruples → blocks, dominators, loops, liveness	M2	219
Cost model	`energy_model.cpp`	Quad → pJ per execution	M2	96
Frequency estimation	`frequency.cpp`	CFG → block and call frequencies	M3·M4	229
Energy analysis	`analysis.cpp`	Frequencies + costs → energy report, hot set	M3	226
Optimisation passes	`optimize/*.cpp`	IR + hot set → rewritten IR	M3	581
Interpreter and profiler	`interpreter.cpp`	IR → output, exit code, per-quad counts	M4	325
Reports and CLI	`report.cpp`, `main.cpp`	Report → text, JSON, DOT; command line	M4	428
Total				**4,132**

5.3 Core data structures

- **Token and AST.** A token carries its kind, lexeme, literal value and line and column. AST nodes are annotated by semantic analysis with types, constant values and unique names (shadowed locals become `x.1`, `x.2`).
- **Quad and Operand.** The IR is a flat vector of quadruples (`op, dst, a, b, c`) with a source line per quad. Operands are variables, temporaries, integer or double constants, labels, functions or strings. Arrays are named objects accessed only by `Load` and `Store`.
- **Block, Loop and CFG.** A block is a range of quads with successor and predecessor lists. A loop records its header, member blocks, latches, exiting blocks, nesting depth and parent. The CFG holds the dominator relation as a bit matrix.
- **EnergyReport.** Energy per block, loop, function, source line and operation class, in a form that the text, JSON and DOT renderers share.
- **HotSet and PassStats.** The set of hot lines and loops that restricts the optimiser, and a per-pass change count with one log line per rewrite.

5.4 Design decisions

Table 5. Key design decisions and why they were taken.

Decision	Rationale
Accept only real C++	g++ becomes a free, independent oracle for differential testing. This replaced the custom language proposed at Review 1.
Quadruple IR with a source line on every quad	Per-line energy attribution is then a by-product of the analysis, with no extra bookkeeping.
Cost model separate from the analysis	The table can be replaced from a file (`--costs`) without touching code, which is the hook for calibration.
Profile-directed passes	Only the lines covering 90% of the energy are rewritten, so effort goes where it can matter and cold code is left alone.
The interpreter is both profiler and oracle	One component supplies exact execution counts and the output used to check every optimisation.
No pointers or array parameters	Distinct arrays cannot alias, which is what makes redundant-load elimination sound without a separate alias analysis.
The web interface wraps the CLI	The browser tool exercises the same code path as the tests, and the JSON report is the only contract between them.

5.5 Command-line interface

Table 6. Command-line options and exit codes.

Option	Effect
`--static`, `--dynamic`, `--both`	Static estimate (default), measured profile, or both with a hotspot-agreement line.
`--opt[=LIST]`, `--opt-all`	Run passes (`sr,cse,licm,rle,dce` or `all`), re-profile and verify. `--opt-all` treats every line as hot.
`--coverage X`	Hot set = lines covering fraction X of energy (default 0.90).
`--costs FILE`	Override any entry of the cost table.
`--ir`, `--ir-opt`, `--cfg`, `--tokens`, `--ast`	Print the internal representations.
`--dot DIR`, `--json FILE`	Write Graphviz CFGs shaded by energy; write a machine-readable report.
`--run`, `--run-opt`	Execute the (optimised) IR and print only the program output.
Exit codes	0 success; 1 compile error; 2 runtime error; 3 optimisation changed the output; 64 usage error.

6. Algorithms and Methodology

6.1 Lexical analysis

The scanner is hand-written and uses maximal munch: each branch of the scan loop is the deterministic automaton for one token class (identifiers and keywords, integer and floating-point numbers with exponents, string and character literals, comments, and one- or two-character operators). It reports an unterminated string or comment, an invalid escape, an integer literal that overflows 32 bits and any unknown character, each with a line and column. Preprocessor lines are skipped. Time is linear in the input.

6.2 Parsing

The parser is recursive descent with one function per non-terminal of the grammar in Appendix D. Operator precedence follows C++ and is encoded by the nesting of the expression functions (||, &&, equality, relational, additive, multiplicative, unary, postfix). On a syntax error it records the diagnostic, skips to the next `;` or `}` (panic mode) and continues, so several errors are reported in one run. A set of keywords that belong to C++ but not to the subset (`class`, `switch`, `template`, `cin`, and others) produces a specific "outside the subset" message rather than a generic one.

6.3 Semantic analysis

A stack of scopes forms the symbol table, searched from the innermost outwards. The pass resolves every name, gives shadowed locals unique names, folds `const` scalars, and checks types and context: implicit `int`, `double` and `bool` conversions; `%` on integers only; assignment to arrays and constants; calls with the wrong number of arguments; use before declaration; `void` used as a value; division by a constant zero; a constant array index that is out of bounds; array sizes that are not constants; and a missing `main`. All errors of the phase are collected before it stops.

6.4 Intermediate code

Code generation is syntax-directed (Dragon Book, chapter 6). Logical && and || become short-circuit control flow. Array reads and writes are explicit `Load` and `Store` quads so that memory traffic is visible to the cost model. An assignment retargets the quad that produced the value, so `x = a + b` is one quad rather than two. Implicit conversions become `I2D` and `D2I`. Listing 1 shows the IR of the `hoist` benchmark, whose loop recomputes `n * 2 + n / 2` on every iteration.

```
func main() -> int
    n = 1000
    i = 0
L1:
    t1 = i < n
    iffalse t1 goto L2
    t2 = n * 2
    t3 = n / 2
    t4 = t2 + t3
    @a[i] = t4
    i = i + 1
    goto L1
    ...
```

Listing 1. Three-address code for `benchmarks/hoist.cpp`.

6.5 Control-flow graph, dominators, loops and liveness

Leaders are the first quad, every label, and every quad after a jump or return; a block runs from one leader to the next. Edges follow the branch targets, and reachability is found by search from the entry. Dominators are computed by the iterative data-flow method on a bit matrix: the entry is dominated only by itself, and every other block starts dominated by all blocks and is repeatedly intersected over its predecessors until nothing

changes. A back-edge is an edge `n → h` where `h` dominates `n`; the natural loop is `h` plus every block that can reach `n` without passing through `h`. Loops are sorted innermost first and nested by strict containment. Liveness is the standard backward data flow, with `in[B] = use[B] ∪ (out[B] − def[B])`. The cost of the dominator step is the number of iterations times the edges times the blocks, which is small for the program sizes in scope.

6.6 Energy model

The energy of a program is the sum, over basic blocks, of the block's execution frequency times the cost of the quads it contains:

$$E = \Sigma_{\text{blocks } b} \; freq(b) \times \Sigma_{q \in b} \; cost(category(q))$$

Each quad maps to a category (for example `mem.load`, `int.div`, `branch.cond`), and each category has a cost in model picojoules (Table 7). The values encode an ordering, memory ≫ divide > multiply > ALU > move > branch, taken from the instruction-level literature [1], [2]. They are **relative**, not measured joules: they are meant to rank code and compare variants. Any entry can be overridden from a text file with `--costs`.

Table 7. Default cost table (model pJ per execution).

Category	Cost	Operations
mem.load	10.0	Load: array read, including address generation
mem.store	12.0	Store
int.alu	1.0	Integer add, subtract, negate, not, comparisons
int.shift	1.0	Shl
int.mul	3.0	Integer multiply
int.div	18.0	Integer divide and modulo (multi-cycle)
fp.alu / fp.mul / fp.div	2.5 / 3.5 / 14.0	Double add and compare / multiply / divide
convert	2.0	Int to double and back
move	0.8	Copy
branch.uncond / branch.cond	0.6 / 0.9	Jump / conditional jump
call / ret / param	6.0 / 4.0 / 0.8	Call overhead
io	0.0	Output (out of scope)

6.7 Execution frequency

Static estimate

For a function invoked once, a block `b` is estimated to run `freq(b) = Π over loops L containing b of (trip(L) + [b is the header of L]) × p(b)`. The trip count is derived from the loop's induction variable: the header must end in a comparison `iv op bound`, `iv` must be a local updated exactly once in the loop by a constant step, and the start and bound must be known constants (including `const` globals and locals assigned once). With constants, the count is exact; for example `i from 0 step 1 vs 1000` gives 1,000. If only the bound is known (a triangular inner loop whose start is another loop's variable) the estimate is `bound / 2`. Otherwise a default of 10 is used. The branch probability `p(b)` is 1 if `b` dominates every latch of its innermost loop (or every return, outside loops) and 0.5 otherwise. Function invocation counts are propagated over the call graph from `main` in topological order, and recursive functions are multiplied by a factor of 10 because their depth is unknown.

Dynamic profile

The interpreter executes the IR and counts how often every quad runs. The frequencies are then exact for the program's single, deterministic run, and a loop's average trip count is the number of back-edge traversals divided by the number of times the loop is entered. With `--both` the tool reports both estimates and whether their top-three hotspots agree.

6.8 Aggregation and the hot set

Block energy is the sum of `cost(q) × executions(q)` over the block. Loop energy is the sum over its blocks and so includes inner loops. Function energy is the sum over its own blocks, and callee cost appears through the callee's invocation count. Energy is also bucketed by source line and by operation class, and the lists are sorted to give the rankings. The **hot set** is selected greedily: lines are taken in decreasing order of energy until their running total reaches the coverage fraction (default 0.90) of the program total, and a loop is hot if its inclusive energy is at least `1 − coverage` of the total.

6.9 Optimisation passes

Every pass rewrites only quads on hot lines (or hot loops). Each states its legality conditions below; none changes the program's observable behaviour.

Strength reduction (SR)

Rewrites `x * 2^k` to `x << k` for integers, `x * 2.0` to `x + x` and `x / 2^k` to a multiplication by an exact power-of-two reciprocal for doubles, removes identities (`x * 1`, `x / 1`, `x + 0`, `x - 0`) and folds operations on two integer constants with 32-bit wrap-around (never dividing by zero). Integer division by a power of two is deliberately **not** turned into a shift: for negative operands C++ truncates toward zero while an arithmetic shift rounds toward negative infinity.

Common-subexpression elimination (CSE)

A value-numbering pass over extended basic blocks. Within a block, `t2 = a op b` is removed when an earlier `t1 = a op b` is still available, meaning neither operand has been redefined since; uses of `t2` are renamed to `t1`. A block with exactly one predecessor, reached by a forward edge, inherits that predecessor's available set, so the scope is a tree of blocks; at joins and loop headers the set starts empty. Calls kill expressions on global variables. This is safe because both temporaries have a single definition and the first definition dominates the second. Loads are not handled here.

Loop-invariant code motion (LICM)

For each hot natural loop, innermost first, an instruction `d = a op b` is hoisted into a new pre-header when it is pure and cannot trap (integer division only by a constant other than 0 and −1), every operand is a constant or has no definition inside the loop (or is itself hoisted), and either `d` is a single-definition temporary, or it is a local defined once in the loop that is not live on entry to the header and that is either defined in a block dominating every exit or dead at every exit. The last condition keeps zero-trip loops correct. Jumps from outside the loop are redirected to the pre-header, and the process repeats until no more code moves. Figure 2 shows the effect on `hoist`.

Redundant-load elimination (RLE)

The pass tracks which array elements currently have a known copy in a scalar name or constant, as a map from (array, index operand) to a value operand. A load adds its destination; a store adds its stored value, which gives store-to-load forwarding. A later load of the same array and index operand is then redundant: it becomes a move, or, when both the destination and the value are single-definition temporaries or constants, it disappears and its uses are renamed. An entry is dropped when its index or value operand is redefined; a store drops every entry of the same array except entries whose constant index provably differs; a call drops all global-array entries and entries that use global scalars. Because the subset has no pointers or array parameters, distinct arrays never alias, so nothing else can invalidate an entry. The scope is the extended basic block, as for CSE. A removed load cannot hide a runtime error, because an identical load with the same index executed earlier on the same path.

Dead-code elimination (DCE)

Three clean-ups. Unreachable blocks are deleted; they execute zero times, so this does not change energy and it is applied to every function. A dead assignment, one whose result is a local or temporary that is not live afterwards, is deleted, repeatedly, so whole chains disappear; only hot lines are touched. Self-moves are deleted. Removable operations are the pure ones (a conversion from double to int is excluded), loads, double

division, and integer division or modulo by a constant other than 0 and −1. A dead load can only trap on a bad index; WattWise optimises a program only after a run of the original has completed without a runtime error, and the subset has no input, so the removed load never trapped. Stores, calls, output, branches and anything that defines a global are never removed.

Pass order

The default order is `sr, cse, licm, cse, rle, dce`. Strength reduction runs first so that later passes see the simplified operations. CSE unifies index temporaries so that RLE's keys match across the compare-and-swap blocks of a sort. LICM follows, and the second CSE removes duplicates it exposes. RLE turns the redundant loads into moves, and DCE runs last to remove whatever the earlier passes left dead.

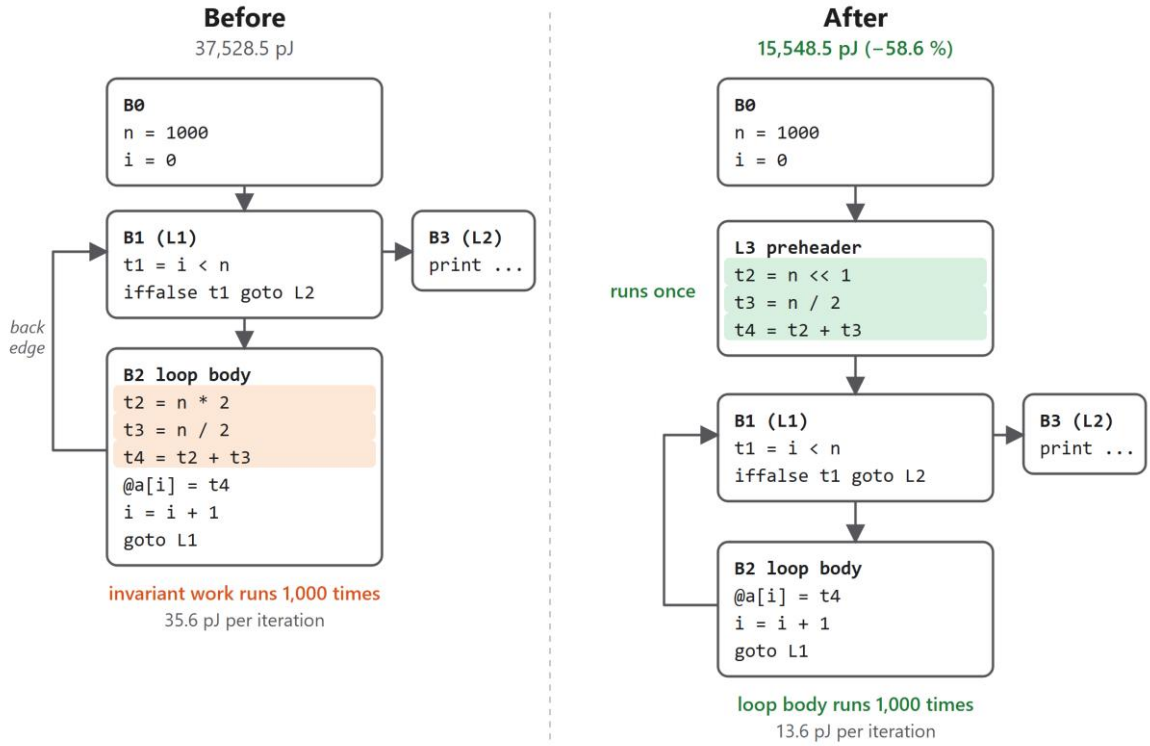


Figure 2. Loop-invariant code motion on benchmarks/hoist.cpp. The three invariant quads move into a pre-header and strength reduction turns the multiply into a shift.

6.10 Verification

Optimisation works on a copy of the IR. The CLI first interprets the original (a failure here stops with exit code 2), applies the passes, re-analyses, interprets the optimised IR and compares the output text and exit code. A difference is reported and the process exits with code 3. Separately, `scripts/difftest.sh` compares g++, the unoptimised IR and the IR optimised with every pass on every line (Section 8).

Table 8. Asymptotic cost of each stage, for n quads, b blocks and e edges.

Stage	Cost	Stage	Cost
Lexing, parsing, semantic analysis	O(n)	Dominators	O(iterations × e × b)
IR generation	O(n)	Natural loops	O(b × e)
Static frequency	O(b + loops × n)	Energy aggregation	O(n)
Strength reduction	O(n)	CSE	O(n log n)
RLE	O(n × tracked entries)	DCE	O(n × rounds)
LICM	O(rounds × loops × n)	Interpreter	O(executed quads)

7. Implementation Details

7.1 Code organisation

The repository is organised by purpose: `src/` and `include/wattwise/` for the compiler (4,132 lines), `src/optimize/` for the five passes, `tests/unit/` for the Catch2 tests (713 lines), `tests/valid/` and `tests/invalid/` for the test programs, `benchmarks/` for the 15 benchmarks with golden outputs, `scripts/` for the two test drivers, `docs/` for the grammar, cost model and IR format, and `webapp/` for the browser interface (1,750 lines of JavaScript, HTML and CSS).

7.2 Build process

```
cmake -S . -B build -DWATTWISE_TESTS=ON
cmake --build build -j
ctest --test-dir build --output-on-failure   # unit + differential + invalid
```

CMake builds three targets: the `wattwise_core` library, the `wattwise` command, and `wattwise_tests`, which links the same library. Compilation uses `-Wall -Wextra -Wpedantic` and the build is warning-free.

7.3 The interpreter

The interpreter resolves variable names to slots and array names to indices once, then executes quads from a flat vector, incrementing a per-quad counter on each step. Integers are 32-bit with wrap-around. It traps on integer division or modulo by zero, an out-of-range array index, a double-to-int conversion out of range, a step count above 2,000,000,000 (a likely infinite loop), a call depth above 10,000, and running off the end of a function, and it reports each with the function name. Those traps are what make the dynamic mode a trustworthy oracle: unlike a native binary, a bad index fails deterministically with a clear message.

7.4 Continuous integration

A GitHub Actions workflow configures, builds and runs `ctest` on Ubuntu 24.04 with GCC 13.3.0 on every push. The latest run on the final commit (`ea51e67`) passed all three suites. The first CI runs had failed before executing any test, because the two test scripts had been committed from Windows without the executable bit; the fix is described in Section 8.4.

7.5 Web interface

The browser interface lets a user paste or load a program, choose whether to optimise and whether to run a dynamic profile, and read the report. A small Express server exposes three endpoints: `GET /api/benchmarks` and `GET /api/benchmarks/:name` (the name is validated against a pattern to prevent path traversal) and `POST /api/analyze`, which writes the source to a temporary file, runs `wattwise` with `--json` through `execFile` (no shell) under a 10-second timeout, removes the temporary files and returns the exit code, the text report and the parsed JSON.

The front end is a single page with no framework and no external requests. It leads with one headline number and a verified-output badge, then shows the before/after energy by operation class, the hotspot and loop tables (with trip counts labelled exact, approximate or guess), and the list of rewrites. Motion uses physically derived spring easing, only `transform` and `opacity` are animated, and reduced-motion, reduced-transparency and high-contrast preferences are honoured. A draggable divider resizes the source and report panes with rubber-banding at the limits. It also handles the case where an optimisation changed the output (exit code 3) by showing the numbers together with a warning.

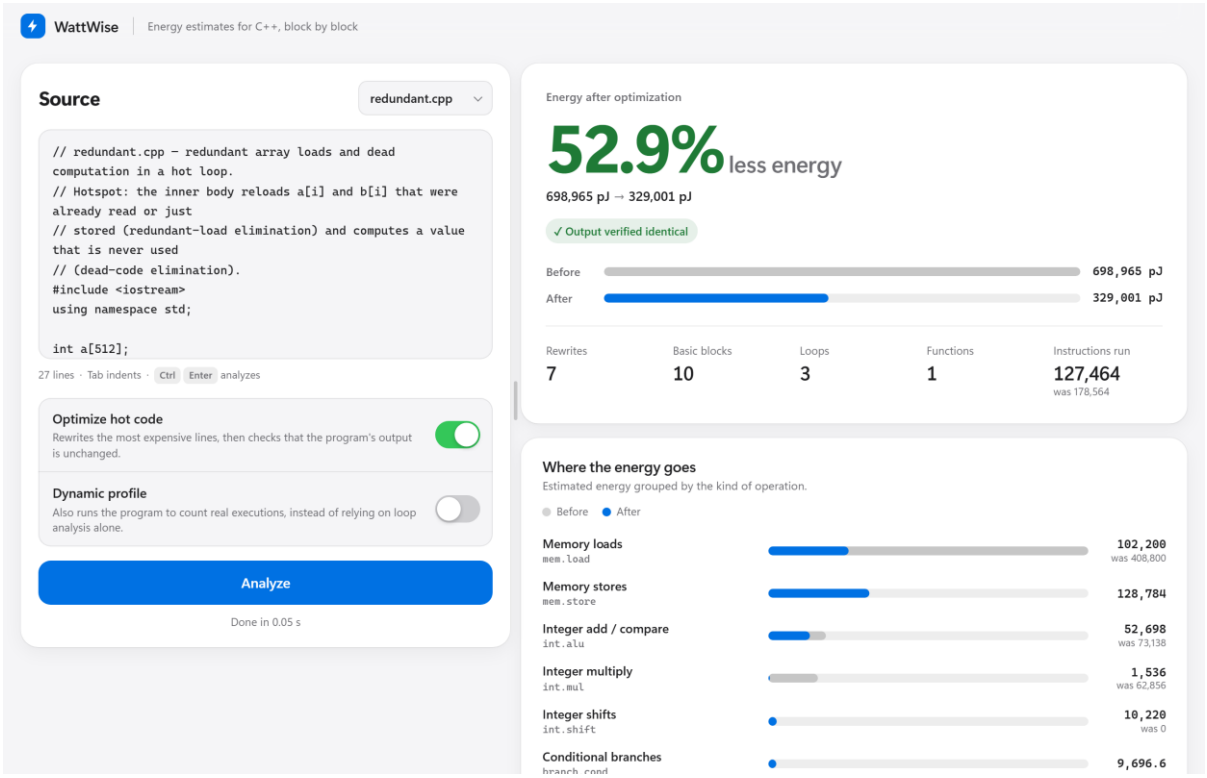


Figure 3. The web interface after analysing redundant.cpp with optimisation on.

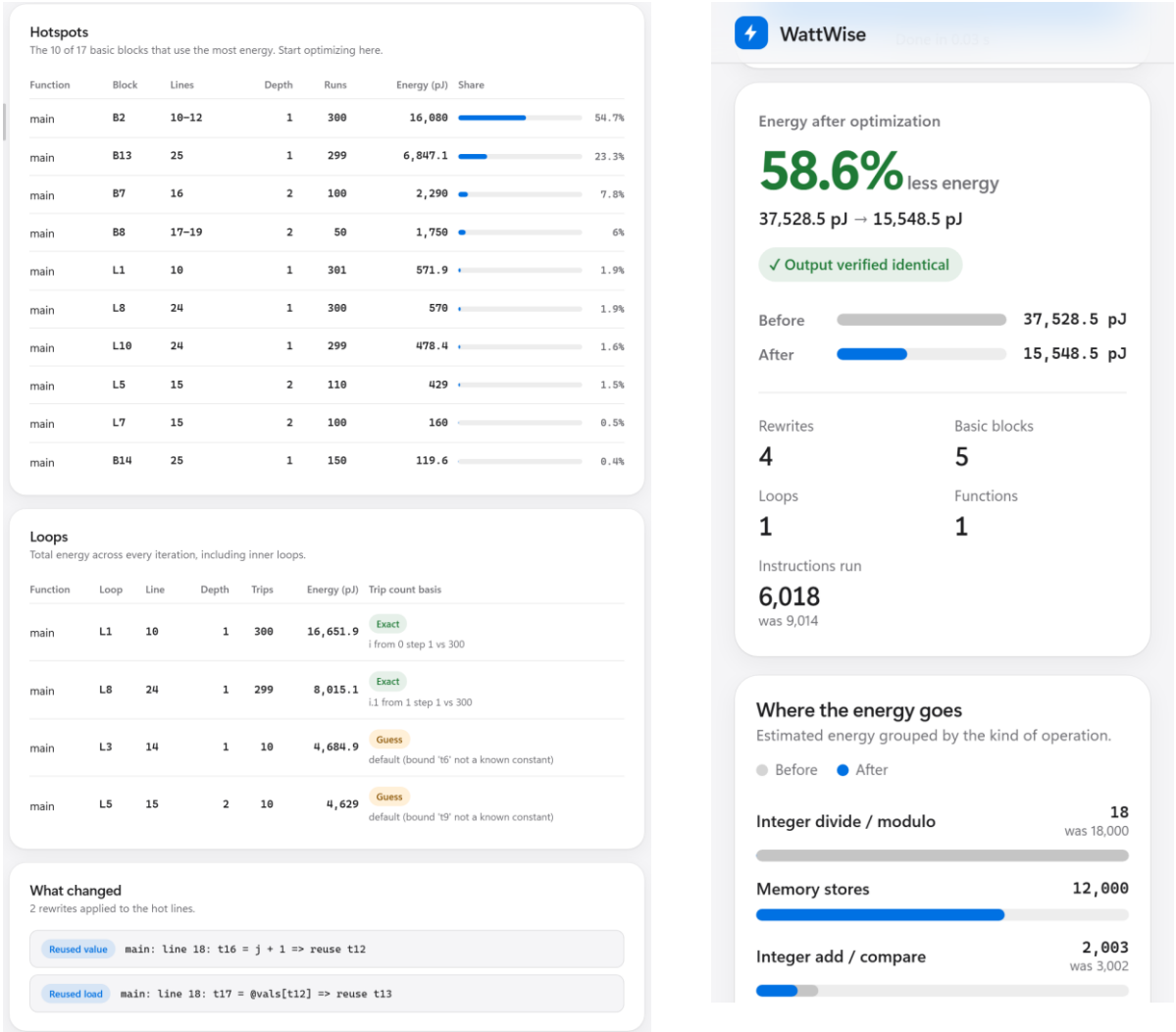


Figure 4. Left: hotspot and loop tables and the list of rewrites for bubble_sort. Right: the phone-width layout.

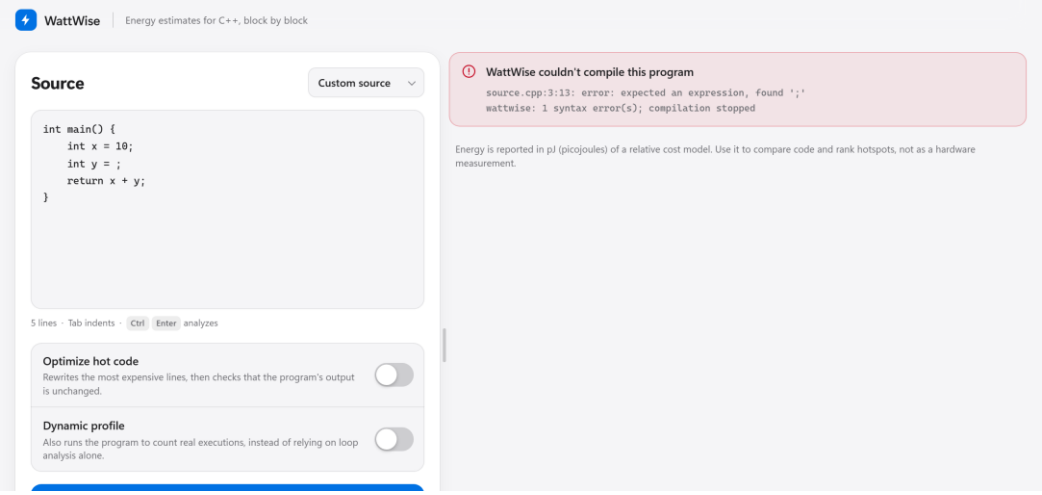


Figure 5. A compile error is shown with a plain title and the compiler's located message.

8. Testing Strategy and Test Cases

8.1 Layers of testing

Table 9. Test layers and results on the final commit.

Layer	What it checks	Size	Result
Unit tests (Catch2)	Lexer, parser, semantic analysis, IR, CFG, energy model and every pass in isolation.	58 cases, 220 assertions	All pass
Invalid programs	Each phase rejects bad input with the right diagnostic phase, exit code 1, and no crash.	33 programs: 9 lexical, 10 syntax, 14 semantic	33 of 33
Differential	g++, unoptimised IR and fully optimised IR give the same output and exit code.	35 programs: 15 benchmarks and 20 others	35 of 35
Continuous integration	The same suites on a clean Ubuntu 24.04 machine with GCC 13.3.0.	Every push	Passing

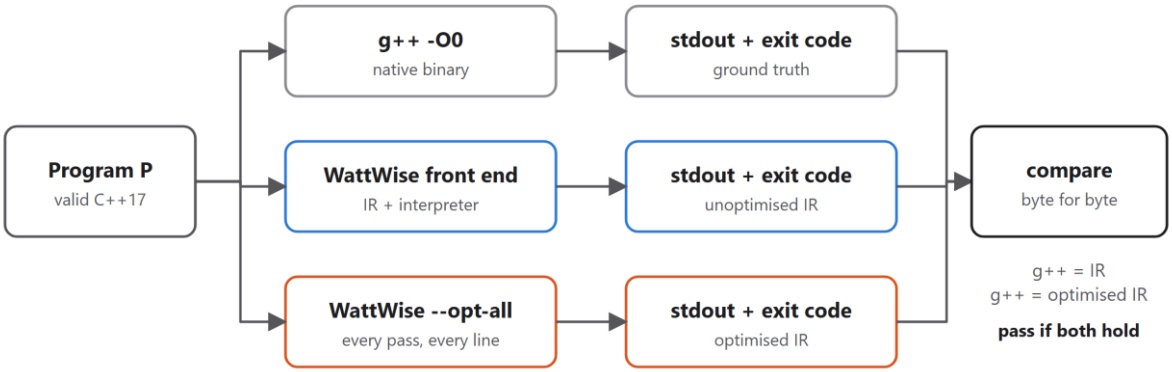


Figure 6. Differential testing. The optimised run applies every pass to every line, the strictest setting.

The differential suite uses `--opt-all`, which treats every line as hot, so it exercises the passes far more aggressively than a normal run does. The three-way comparison separates fault locations: a mismatch between g++ and the unoptimised IR points to the front end, IR generator or interpreter, and a mismatch between the unoptimised and optimised IR points to a pass.

8.2 Representative test cases

Table 10. Selected test cases with expected and observed results.

#	Test (input)	Expected	Observed
1	Lexical: unterminated string literal	Lexical error, exit 1	`4:22: error: unterminated string literal`; exit 1
2	Syntax: missing `;` after a declaration	Syntax error, exit 1	`2:24: error: expected ';' after declaration, found 'return'`; exit 1
3	Semantic: use of an undeclared name	Semantic error, exit 1	`2:21: error: use of undeclared identifier 'y'`; exit 1
4	Boundary: `10 / 0` (g++ accepts it)	Rejected by a subset rule	`2:27: error: division by constant zero`; exit 1
5	Failure: `a[i]` with `i = 10`, size 4, run with `--dynamic`	Runtime error, exit 2	`array index 10 out of bounds (size 4)`; exit 2. A native g++ binary gives undefined behaviour.
6	Program returning 42 (`exit_code.cpp`)	Same exit code in all three runs	g++, IR and optimised IR all exit 42
7	SR on negative operands: `-7*4`, `-7/2`, `-7%2`	`-28 -3 -1` (C++ semantics)	Identical after the pass
8	LICM: `10 / z` inside a loop that runs zero times	No trap; output 0	Output 0; division not hoisted
9	RLE: store to the same array between two loads, index aliasing and not	`6 99` and `6 6`	Both as expected; second load kept
10	RLE: call between loads of a global and a local array	Global reload kept, local reload removed	Output `1 50 2 2`; one load removed
11	DCE: dead `int u = 10 / z` with `z = 0`	Still traps	Interpreter still fails after the pass
12	DCE: code after `return`, with an empty hot set	Unreachable code removed; exit code kept	Code removed; exit code 3
13	Cold code under profile direction (empty hot set)	No rewrite	0 changes
14	`redundant.cpp` with `--opt`	Output identical; energy falls	`total=3049800` unchanged; −52.9%

8.3 Boundary, failure and stress cases

Boundary cases include zero-trip loops, negative integer arithmetic, loops with unknown bounds, recursion, empty hot sets and arrays whose index is a constant at the edge of the array. Failure cases are the thirty-three invalid programs plus runtime traps. Stress is bounded by the interpreter's limits (2×10^9 steps, depth 10,000); the longest benchmark executes 2.5 million IR instructions (`collatz`), and the whole pipeline takes a mean of 15 ms per benchmark (Section 9.5).

8.4 Regression and what the tests caught

- **Mutation check.** To confirm the RLE tests can fail, the store-invalidation rule was disabled on purpose. Three assertions failed, including a wrong program output, and the rule was restored.
- **Continuous integration.** The first three CI runs failed before running any test because `difftest.sh` and `invalidtest.sh` had mode 100644. The fix was a mode-only change (`100644` to `100755`); the next run passed all three suites.
- **Environment.** On Windows the `.sh` scripts cannot be started by `ctest` directly, and when run from Git Bash every invocation of the binary failed with "command not found" (exit 127) until the MSYS2 toolchain directory was placed first on PATH, which points to a conflicting runtime library earlier on the default PATH.

9. Results and Evaluation

9.1 Correctness

All 35 valid programs produce the same output and exit code under g++, the unoptimised IR and the IR optimised with every pass on every line. All 33 invalid programs are rejected in the expected phase, none crashes the compiler, and fifteen of them are accepted by g++ 13.2 (`-fsyntax-only`) and rejected here by design, either as subset rules or because g++ only warns. All 58 unit tests pass. Every one of the 15 normal optimisation runs finished with the verification line reporting identical output and exit code 0.

9.2 Optimisation effectiveness

Table 11 lists the result for each benchmark. The static columns use the default profile-directed run; the dynamic column repeats it using the interpreter's measured counts, so its numbers are exact with respect to the cost model. The last two columns give the IR instructions the interpreter executed before and after, and the number of rewrites by pass (SR, CSE, LICM, RLE, DCE).

Table 11. Results per benchmark, sorted by static reduction. Energy in model pJ.

Benchmark	Blocks / loops	Static before	Static after	Static Δ	Dynamic Δ	IR instructions run	Rewrites by pass
hoist	4 / 1	37,529	15,549	58.6%	58.6%	9,014 → 6,018	1/0/3/0/0
redundant	10 / 3	698,965	329,001	52.9%	52.9%	178,564 → 127,464	1/0/0/2/4
binsearch	20 / 3	2,619,900	2,119,900	19.1%	18.9%	973,850 → 922,105	0/0/0/1/0
stencil	16 / 5	67,610	60,202	11.0%	15.2%	498,493 → 457,736	1/0/4/0/0
matmul	19 / 6	548,832	505,128	8.0%	8.0%	197,713 → 183,361	0/2/2/0/0
histogram	17 / 2	2,757,160	2,577,160	6.5%	7.7%	479,924 → 469,847	0/1/0/0/0
integrate	5 / 1	806,060	786,059	2.5%	2.5%	340,017 → 340,017	1/0/0/0/0
bubble_sort	17 / 4	29,932	29,382	1.8%	24.4%	729,972 → 684,400	0/1/0/1/0
poly_eval	15 / 4	397,248	397,248	0.0%	3.9%	150,812 → 150,812	0/0/0/0/0
collatz	13 / 2	1,099,740	1,099,740	0.0%	0.0%	2,484,409 → 2,484,409	1/0/0/0/0
fib_recursive	7 / 1	2,916	2,916	0.0%	0.0%	247,889 → 247,889	0/0/0/0/0
gcd_lcm	12 / 3	869,420	869,420	0.0%	0.0%	154,022 → 154,022	0/0/0/0/0
prefix_sum	12 / 3	419,300	419,300	0.0%	0.0%	126,230 → 126,230	0/0/0/0/0
sieve	9 / 2	1,916,910	1,916,910	0.0%	0.0%	406,485 → 406,485	0/0/0/0/0
triangular	14 / 3	2,361,170	2,361,170	0.0%	0.0%	1,167,002 → 1,167,002	0/0/0/0/0

Across all 15 benchmarks the mean static reduction is **10.7%** and the mean dynamic reduction **12.8%**; 8 programs improve under the static model and 9 under measured counts. The largest gains are on `hoist` (58.6%, loop-invariant work), `redundant` (52.9%, repeated loads and dead code) and `binsearch` (19.1%, a repeated load of the middle element). Seven programs are unchanged. For `prefix_sum` the reason is that its redundancy is loop-carried (`pre[i] = pre[i-1] + …` reads a value stored under a different index on the previous iteration), which the extended-block RLE does not forward; `sieve` only stores. The remaining zeros have not been analysed individually; for each of them the current passes find no change that lowers the estimated energy of the hot lines.

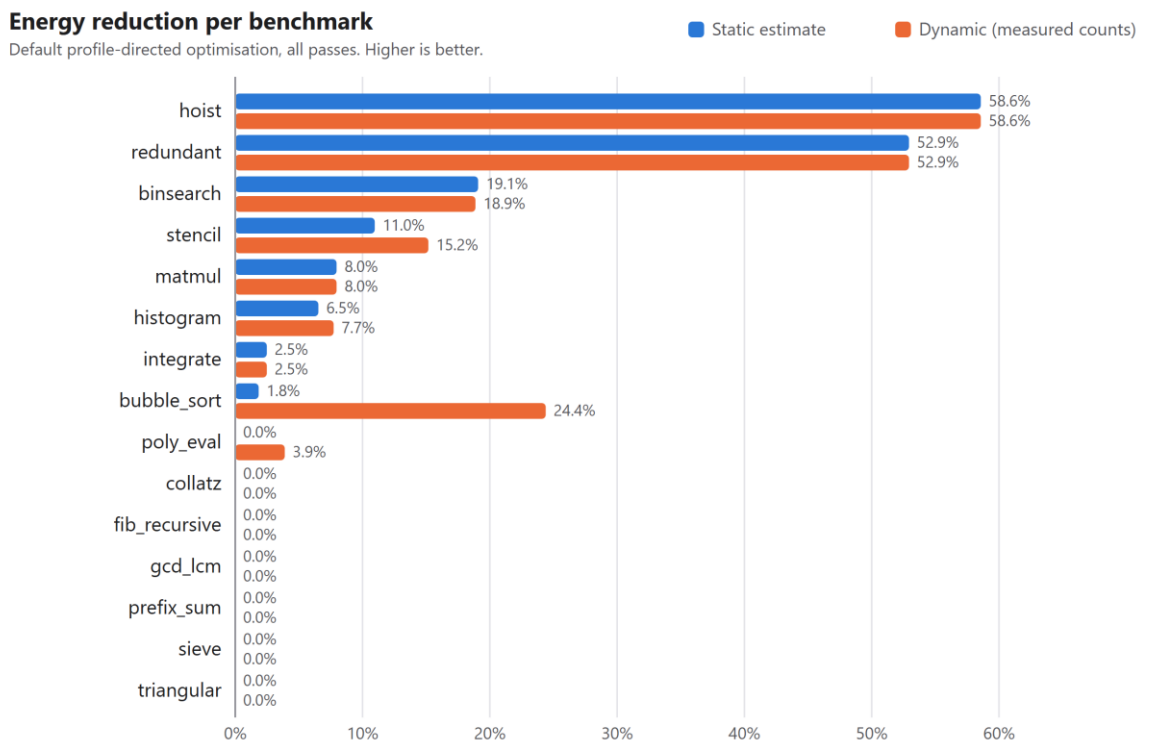


Figure 7. Energy reduction for each benchmark, static estimate against measured counts.

9.3 What the Review 3 passes add

At the start of the Review 3 work the optimiser had strength reduction, block-local CSE and LICM. On the 14 benchmarks that existed then, it improved 5 of 14 programs with a mean static reduction of 6.2%. With redundant-load and dead-code elimination, and with CSE extended to extended basic blocks, the same 14 programs now give 7 of 14 improved and a mean of 7.7% (9.9% under measured counts). Figure 8 isolates the contribution by running the older pass list (`sr,cse,licm,cse`, here already using the extended-block CSE) against the full list.

Table 12. Summary before and after the Review 3 passes.

Measure	Start of Review 3 work (14 programs)	Final, same 14 programs	Final, all 15 programs
Mean static reduction	6.2%	7.7%	10.7%
Programs improved (static)	5 of 14	7 of 14	8 of 15
Mean dynamic reduction	not recorded	9.9%	12.8%

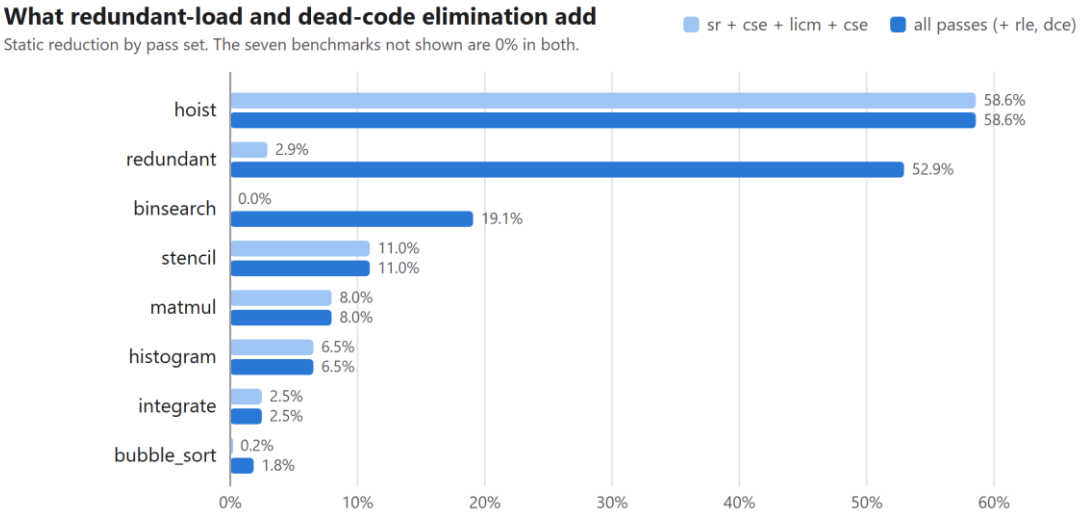


Figure 8. Static reduction with the older pass list and with all five passes. The seven programs not shown are 0% in both.

The largest change is on `redundant`, which rises from 2.9% to 52.9% once RLE and DCE are added, and on `binsearch`, which rises from 0% to 19.1%. Figure 9 shows why for `redundant`: memory-load energy falls from 408,800 to 102,200 pJ, integer-multiply energy from 62,856 to 1,536 pJ because a dead chain is removed, and register-move energy rises from 19 to 8,195 pJ because each removed load leaves a cheap move behind. The optimiser has traded expensive operations for cheap ones, as intended.

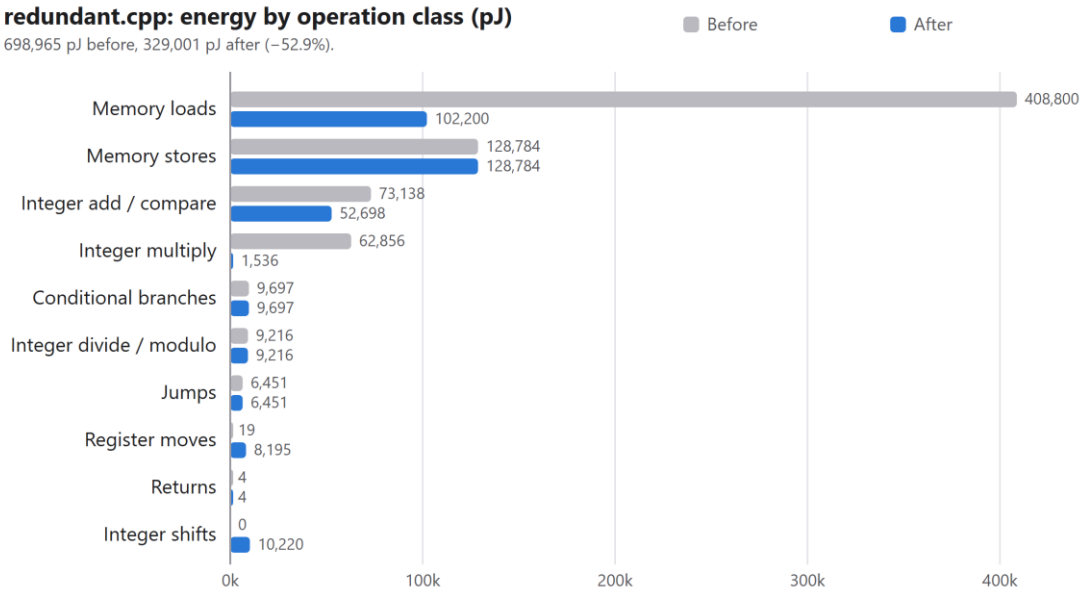


Figure 9. Energy by operation class for redundant.cpp before and after optimisation.

9.4 Static estimate against measured counts

With `--both` the tool reports whether the static and dynamic rankings agree. They agree closely on which block is hottest: the top block is the same on **14 of 15** programs and the full top-three set matches on **12 of 15**. The exceptions are `bubble_sort` (top block differs, one of three in common), `poly_eval` and `stencil` (two of three in common). The ranking, which is what drives hot-set selection, is therefore reliable on this benchmark set.

The magnitude is a different matter. Figure 10 divides the measured total by the static total. Where every loop bound is a constant (`hoist`, `integrate`, `matmul`, `redundant`) the two are identical to three significant figures. Where a bound is unknown the static estimator falls back to a default trip count of 10 and its error is large: it is 78 times too low for `bubble_sort` (inner bound `n - 1 - pass`), 172 times too low for `fib_recursive` (recursion is weighted ×10 rather than by true depth), 30 times too low for `stencil`, and 7.3 times too low for `collatz` (a data-dependent loop). It can also be too high, as for `gcd_lcm` (0.41×) and `sieve` (0.45×).

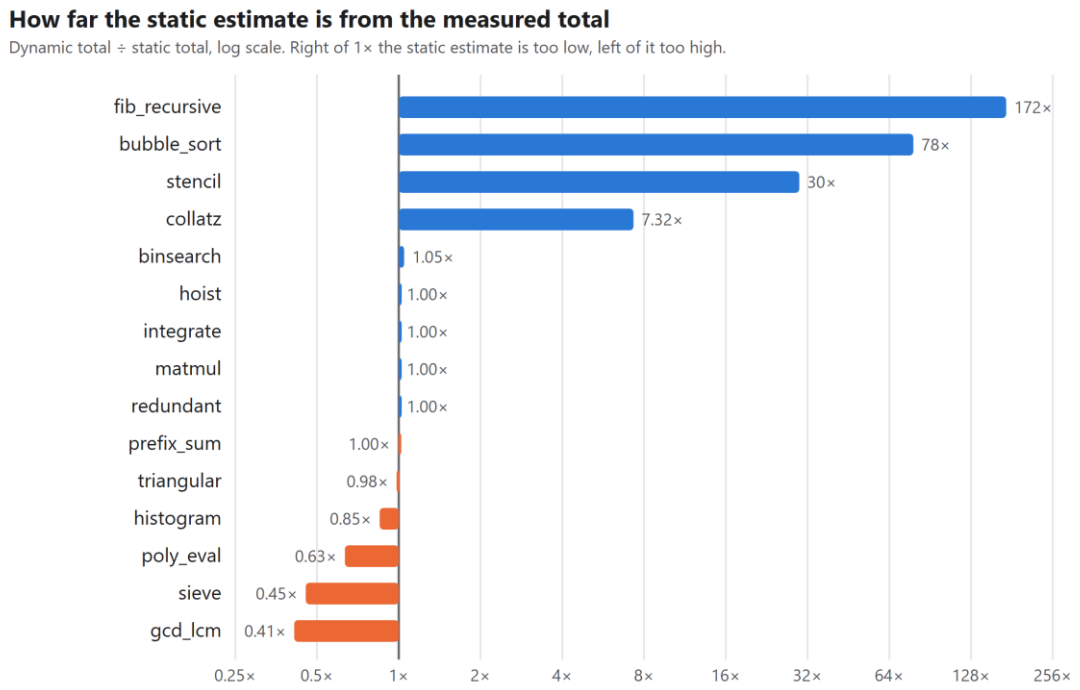


Figure 10. Measured total divided by static total on a logarithmic scale.

This has a visible consequence for optimisation. The static model credits `bubble_sort` with a 1.8% reduction, but under measured counts the same kind of rewrite saves 24.4%: the swap block (lines 17 to 19) is estimated to run 50 times statically and is measured at 22,786 executions, about 456 times more. Likewise `poly_eval` shows 0% statically and 3.9% dynamically. Its hot set depends on the frequencies: with measured counts it reaches `main` lines 30 and 31, where `x / 4.0` appears twice, so strength reduction rewrites both to `x * 0.25` and CSE then reuses one of them; with static frequencies those lines fall below the 90% cutoff and are left alone. The practical advice is to use the static mode to rank and the dynamic mode to size the benefit.

9.5 Cost of the analysis

Running the full pipeline on a benchmark, including parsing, IR generation, analysis, all five passes and two interpreter runs, took a mean of 15 ms and at most 27 ms per program on the development machine (wall time of the process, including start-up). The interpreter is the dominant cost. At these sizes analysis time is not a concern.

9.6 Threats to validity

- **The model is relative.** Percentages are reductions in model picojoules. They say which variant is cheaper under the model, not how many joules a device would save.
- **No microarchitecture.** A load costs the same whether it would hit in cache or miss. Real energy savings from removing a load depend on where it would have been served.
- **Benchmark selection.** The 15 programs were written by the team, several (`hoist`, `redundant`) to exhibit a specific transformation, so the mean reduction is not a prediction for arbitrary code.
- **One machine.** Timing figures come from one Windows development machine.

10. Error Handling, Limitations and Known Issues

10.1 Error handling

Table 13. Error classes, diagnostics and exit codes.

Class	Examples	Behaviour	Exit
Lexical	Unknown character, unterminated string or comment, bad escape, integer overflow	Located message; all lexical errors reported; compilation stops	1
Syntax	Missing `;` or brace, dangling operator, `else` without `if`, features outside the subset	Panic-mode recovery; several errors per run; specific message for excluded C++	1
Semantic	Undeclared or redeclared name, type misuse, wrong argument count, constant division by zero, constant index out of bounds	All errors of the phase reported	1
Runtime	Index out of bounds, division by zero, step limit, recursion depth	Function name and cause; program output printed so far	2
Verification	Optimised output differs from the original	Report printed; web interface warns	3
Usage	Unknown option, missing value or file	Usage text	64

10.2 Limitations and known issues

- **Relative cost model.** The constants encode an ordering from the literature and are not calibrated to any processor. There is no cache, pipeline, branch-prediction or frequency-scaling model.
- **Static frequency estimation.** Exact only for loops with constant start, step and bound. Elsewhere it guesses (default trip count 10, branch probability 0.5, recursion ×10) and can be wrong by two orders of magnitude (Section 9.4).
- **Optimiser scope.** Five passes. RLE and CSE work within extended basic blocks and do not forward values across loop iterations; there is no copy or constant propagation and no dead-store elimination. Seven of the 15 benchmarks are unchanged.
- **Language subset.** No pointers, classes, templates, multi-dimensional arrays, array parameters, input or concurrency. The tool cannot be pointed at arbitrary real-world C++.
- **Web server.** Intended for local use. It limits request size and run time but does not sandbox the analysed program beyond the interpreter's step and depth limits, and it has no authentication; it should not be exposed to untrusted users.
- **Windows testing.** `ctest` cannot start the `.sh` test drivers directly on Windows; they must be run from Git Bash with the MSYS2 toolchain first on PATH (Section 8.4). The CI environment is Linux.

11. Member-wise Contribution

The project README assigns each member one module group. The matrix below follows that assignment; the evidence column is to be completed with each member's own commits, pull requests, tests and demonstration artefacts, because the evaluator may inspect the repository history to confirm these claims.

Drafting note, delete before submission: at the time of writing the repository history shows commits from a single account. Link each member's own commits, pull requests and test artefacts below before the review.

Table 14. Final contribution matrix.

Member	Primary technical ownership	Final deliverable produced	Evidence / commit / test	Integration / support	Viva module
Abhyuday Thapliyal (24BCT0204)	M1: lexer, parser, semantic analysis, subset definition	Front end; `docs/subset.md`; invalid-program suite	commits / PRs	integration role	Front end
Arhat Gupta (24BDS0195)	M2: IR, IR generation, CFG, cost model	IR, dominators, loops, liveness; `docs/cost_model.md`, `docs/ir_format.md`	commits / PRs	integration role	IR, CFG, cost model
Anaya Singh (24BCI0077)	M3: analysis, hot-set selection, optimisation passes, static frequency	Energy analysis; SR, CSE, LICM, RLE, DCE	commits / PRs	integration role	Analysis and passes
Keshav Biyani (24BCE0344)	M4: interpreter, reporting, CLI, benchmarks, CI	Interpreter and profiler; text, JSON and DOT reports; CI; test drivers	commits / PRs	integration role	Runtime, testing, release

12. Conclusion and Future Work

12.1 What was achieved

WattWise meets its core objectives. It compiles a subset of C++ to a readable IR, builds the control-flow structures a classical optimiser needs, estimates energy per block, loop, line and operation class under a documented cost model, ranks the hotspots with both a static estimator and exact counts, rewrites the hot code with five passes, and verifies each rewrite by execution. The result is 10.7% mean reduction in estimated energy over 15 benchmarks (12.8% under measured counts), up to 58.6%, with all differential, invalid-program and unit tests passing and continuous integration green.

12.2 What we learned

- A cheap static estimator is good at ranking and poor at magnitude; the right use is to rank statically and size the benefit dynamically.
- An execution oracle changes how optimisations are written: a pass that miscompiles is caught by the next test run instead of by inspection. The mutation check showed the tests can fail.
- Excluding pointers and array parameters from the language bought a simple and sound redundant-load elimination; the cost is generality.
- Small engineering details decide whether a project is reproducible: a missing executable bit failed every CI run until it was found.

12.3 Future work

- **Calibrate the cost table with hardware counters.** On an Intel Linux machine, RAPL could be read around micro-benchmarks that isolate one operation class at a time, with frequency pinned, and the per-operation energies fed to `--costs`. This would replace the ordering taken from the literature with measured constants for one processor. It would not by itself add cache or pipeline effects, which need a memory-hierarchy model.
- **Forward values across loop iterations** (scalar replacement) so that `prefix_sum`-style recurrences stop reloading what the previous iteration stored.
- **More passes:** copy and constant propagation, dead-store elimination.
- **Better static estimates:** symbolic loop bounds, and branch heuristics in the style of Ball and Larus [6], to reduce the gap shown in Figure 10.
- **A cache model**, so that a load's cost depends on locality.
- **A wider language:** multi-dimensional arrays, and pointers with an alias analysis.

Further-development potential. The most defensible outward-facing contribution is as a teaching tool: a complete, inspectable compiler with an energy model and a verification oracle in four thousand lines. A short paper on the gap between static and measured frequency estimation, using the benchmark results here, would be a modest but honest research contribution once the cost model is calibrated.

13. References

[1] V. Tiwari, S. Malik and A. Wolfe, "Power analysis of embedded software: a first step towards software power minimization," IEEE Transactions on VLSI Systems, vol. 2, no. 4, 1994.

[2] S. Steinke, M. Knauer, L. Wehmeyer and P. Marwedel, "An accurate and fine grain instruction-level energy model supporting software optimizations," Proc. PATMOS, 2001.

[3] H. David, E. Gorbatov, U. R. Hanebutte, R. Khanna and C. Le, "RAPL: memory power estimation and capping," Proc. ACM/IEEE ISLPED, 2010.

[4] A. V. Aho, M. S. Lam, R. Sethi and J. D. Ullman, Compilers: Principles, Techniques, and Tools, 2nd ed., Addison-Wesley, 2006.

[5] S. S. Muchnick, Advanced Compiler Design and Implementation, Morgan Kaufmann, 1997.

[6] T. Ball and J. R. Larus, "Branch prediction for free," Proc. ACM PLDI, 1993.

[7] Y. Wu and J. R. Larus, "Static branch frequency and program profile analysis," Proc. IEEE/ACM MICRO-27, 1994.

[8] W. M. McKeeman, "Differential testing for software," Digital Technical Journal, vol. 10, no. 1, 1998.

[9] Catch2 test framework, v3.5.4. https://github.com/catchorg/Catch2

[10] WattWise source repository. https://github.com/abhithapliyal05/WattWise

Verify each citation's details against the source before submission.

Appendix A. User Manual

A.1 Build and run

Requirements: a C++17 compiler (g++ 11 or newer, or clang 14 or newer) and CMake 3.16 or newer. Node.js 18 or newer is needed only for the web interface.

```
cmake -S . -B build -DWATTWISE_TESTS=ON
cmake --build build -j

build/wattwise benchmarks/hoist.cpp                 # static energy report
build/wattwise benchmarks/hoist.cpp --both          # static, dynamic, hotspot agreement
build/wattwise benchmarks/hoist.cpp --opt           # optimise hot code, re-profile, verify
build/wattwise benchmarks/hoist.cpp --ir --cfg      # inspect IR and CFG
build/wattwise benchmarks/matmul.cpp --dot out/     # Graphviz energy heat map
build/wattwise benchmarks/hoist.cpp --opt --json r.json
```

On Windows with MSYS2, use `-G Ninja -DCMAKE_CXX_COMPILER=<path to g++.exe>` and run the shell scripts from Git Bash with the MSYS2 `ucrt64/bin` directory first on `PATH`.

A.2 Web interface

```
cd webapp
npm install
npm start        # then open http://localhost:5173
```

Paste a program or pick a sample, switch on **Optimize hot code** and, if wanted, **Dynamic profile**, and press Analyze (or Ctrl+Enter). The example buttons on the empty screen load a sample and analyse it with optimisation on. The divider between the two panes can be dragged or moved with the arrow keys.

A.3 Tests

```
ctest --test-dir build --output-on-failure
scripts/difftest.sh build/wattwise        # g++ vs IR vs optimised IR
scripts/invalidtest.sh build/wattwise     # every invalid program must be rejected
```

Appendix B. Sample Runs

B.1 IR and control-flow graph of hoist.cpp

```
func main() -> int
    n = 1000
    i = 0
L1:
    t1 = i < n
    iffalse t1 goto L2
    t2 = n * 2
    t3 = n / 2
    t4 = t2 + t3
    @a[i] = t4
    i = i + 1
    goto L1
L2:
    t6 = @a[0]
    print t6
    print " "
    t7 = n - 1
    t8 = @a[t7]
    print t8
    print endl
    return 0
end
```

```
func main: 4 blocks, 1 loop(s)
  B0  quads [0,2)  depth 0  -> B1
  B1 (L1)  quads [2,5)  depth 1  -> B2 B3
  B2  quads [5,11)  depth 1  -> B1
  B3 (L2)  quads [11,20)  depth 0  ->
  loop header B1 depth 1 blocks {B1,B2} latches {B2}
```

B.2 Optimised IR of hoist.cpp

```
func main() -> int
    n = 1000
    i = 0
L3:
    t2 = n << 1
    t3 = n / 2
    t4 = t2 + t3
L1:
    t1 = i < n
    iffalse t1 goto L2
    @a[i] = t4
    i = i + 1
    goto L1
L2:
    t6 = @a[0]
    print t6
    print " "
    t7 = n - 1
    t8 = @a[t7]
    print t8
    print endl
    return 0
end

Verification: optimised program output IDENTICAL (9014 -> 6018 IR instructions executed)
```

B.3 Text report for hoist.cpp with --opt

```
WattWise energy report — benchmarks\hoist.cpp  [static estimate]
Total estimated energy: 37,528.5 pJ (model units; relative, not absolute joules)

Top basic blocks (hotspots)
  #  function    block   lines    depth     executions   pJ/exec       energy pJ   share
  1  main        B2      11-13    1             1,000      35.6        35,600.0   94.9%
  2  main        L1      11       1             1,001       1.9         1,901.9    5.1%
  3  main        L2      15-16    0                 1      25.0            25.0    0.1%
  4  main        B0      9-10     0                 1       1.6             1.6    0.0%

Loops (inclusive energy)
  function    header  line  depth       trip       energy pJ   share   trip-count source
  main        L1      11    1        1,000.0        37,501.9   99.9%   i from 0 step 1 vs 1000

Functions (exclusive energy)
  main          calls 1                  37,528.5  100.0%

Energy by operation class
  int.div                 18,000.0   48.0%
  mem.store               12,000.0   32.0%
  int.alu                  3,002.0    8.0%
  int.mul                  3,000.0    8.0%
  branch.cond                900.9    2.4%
  branch.uncond              600.0    1.6%
  mem.load                    20.0    0.1%
  ret                          4.0    0.0%
  move                         1.6    0.0%

Hottest source lines
  main:12                 34,000.0   90.6%
  main:11                  2,501.9    6.7%
  main:13                  1,000.0    2.7%
  main:15                     21.0    0.1%
```

```
  main:16                     4.0    0.0%
  main:9                      0.8    0.0%
  main:10                     0.8    0.0%

Optimisation (profile-directed: hot lines/loops only, static model)
  strength-reduction  1 change(s)
      main: line 12: t2 = n * 2  =>  t2 = n << 1
  local-cse           0 change(s)
  licm                3 change(s)
      main: line 12: hoisted 't2 = n << 1' out of loop L1 into L3
      main: line 12: hoisted 't3 = n / 2' out of loop L1 into L3
      main: line 12: hoisted 't4 = t2 + t3' out of loop L1 into L3
  local-cse           0 change(s)
  redundant-load-elim 0 change(s)
  dead-code-elim      0 change(s)

  function             before pJ        after pJ     delta
  main                  37,528.5        15,548.5    -58.6%
  TOTAL                 37,528.5        15,548.5    -58.6%

Verification: optimised program output IDENTICAL (9014 -> 6018 IR instructions executed)
```

B.4 Optimisation log for redundant.cpp

```
Optimisation (profile-directed: hot lines/loops only, static model)
  strength-reduction  1 change(s)
      main: line 19: y = t10 * 2  =>  y = t10 << 1
  local-cse           0 change(s)
  licm                0 change(s)
  local-cse           0 change(s)
  redundant-load-elim 2 change(s)
      main: line 17: t8 = @a[i.1]  =>  reuse t7
      main: line 19: t10 = @b[i.1]  =>  reuse x
  dead-code-elim      4 change(s)
      main: line 20: removed dead 'dead = t14 + y'
      main: line 20: removed dead 't14 = t13 * 5'
      main: line 20: removed dead 't13 = @a[t12]'
      main: line 20: removed dead 't12 = i.1 - 1'

  function             before pJ        after pJ     delta
  main                  698,965.0       329,001.0    -52.9%
  TOTAL                 698,965.0       329,001.0    -52.9%

Verification: optimised program output IDENTICAL (178564 -> 127464 IR instructions executed)
```

Appendix C. Complete Test Table

C.1 Invalid programs

Table 15. The 33 invalid programs: expected phase, observed phase, exit code, and whether g++ 13.2 accepts the program.

Program	Expected	Observed	Exit	g++ -fsyntax-only
lex_bad_char	lexical	lexical	1	rejects
lex_bad_escape	lexical	lexical	1	accepts (subset rule or warning only)
lex_bitwise_and	lexical	lexical	1	accepts (subset rule or warning only)
lex_cin_unsupported	lexical	lexical	1	accepts (subset rule or warning only)
lex_int_overflow	lexical	lexical	1	accepts (subset rule or warning only)
lex_shift_right	lexical	lexical	1	accepts (subset rule or warning only)
lex_ternary_unsupported	lexical	lexical	1	accepts (subset rule or warning only)
lex_unterminated_comment	lexical	lexical	1	rejects
lex_unterminated_string	lexical	lexical	1	rejects
sem_arg_count	semantic	semantic	1	rejects
sem_assign_array	semantic	semantic	1	rejects
sem_const_assign	semantic	semantic	1	rejects
sem_div_zero	semantic	semantic	1	accepts (subset rule or warning only)
sem_index_oob	semantic	semantic	1	accepts (subset rule or warning only)
sem_mod_double	semantic	semantic	1	rejects
sem_no_main	semantic	semantic	1	accepts (subset rule or warning only)
sem_nonconst_array_size	semantic	semantic	1	accepts (subset rule or warning only)
sem_redeclared	semantic	semantic	1	rejects
sem_return_value_void	semantic	semantic	1	rejects
sem_subscript_scalar	semantic	semantic	1	rejects
sem_undeclared	semantic	semantic	1	rejects
sem_use_before_decl	semantic	semantic	1	rejects
sem_void_value	semantic	semantic	1	rejects
syn_array_param_unsupported	syntax	syntax	1	accepts (subset rule or warning only)
syn_class_unsupported	syntax	syntax	1	accepts (subset rule or warning only)
syn_dangling_operator	syntax	syntax	1	rejects
syn_else_without_if	syntax	syntax	1	rejects
syn_missing_brace	syntax	syntax	1	rejects
syn_missing_paren	syntax	syntax	1	rejects
syn_missing_semi	syntax	syntax	1	rejects
syn_struct_unsupported	syntax	syntax	1	accepts (subset rule or warning only)
syn_switch_unsupported	syntax	syntax	1	accepts (subset rule or warning only)
syn_template_unsupported	syntax	syntax	1	accepts (subset rule or warning only)

C.2 Differential suite

Table 16. The 35 valid programs: exit code under g++, and agreement of the unoptimised and fully optimised IR with g++.

Program	g++ exit	IR	Opt. IR	Program	g++ exit	IR	Opt. IR
binsearch	0	ok	ok	bool_logic	0	ok	ok
bubble_sort	0	ok	ok	compound_assign	0	ok	ok
collatz	0	ok	ok	const_folding	0	ok	ok
fib_recursive	0	ok	ok	double_array	0	ok	ok
gcd_lcm	0	ok	ok	euler_sum	0	ok	ok
histogram	0	ok	ok	exit_code	42	ok	ok
hoist	0	ok	ok	for_nested	0	ok	ok
integrate	0	ok	ok	global_arrays	0	ok	ok
matmul	0	ok	ok	incdec	0	ok	ok
poly_eval	0	ok	ok	mixed_conversion	0	ok	ok
prefix_sum	0	ok	ok	nested_if	0	ok	ok
redundant	0	ok	ok	print_formats	0	ok	ok
sieve	0	ok	ok	recursion_ackermann	0	ok	ok
stencil	0	ok	ok	recursion_power	0	ok	ok
triangular	0	ok	ok	scopes_shadow	0	ok	ok
arith_double	0	ok	ok	void_function	0	ok	ok
arith_int	0	ok	ok	while_collatz	0	ok	ok
array_init_list	0	ok	ok				

C.3 Unit test cases

Table 17. The 58 Catch2 test cases.

#	Test case	#	Test case
1	strength reduction: x*2 -> x<<1, algebraic identities	30	unknown bound falls back to the default trip count
2	strength reduction preserves negative-number semantics of multiply	31	static energy equals Σ cost × freq and sorts hotspots
3	local CSE removes a repeated expression	32	dynamic profile agrees with static on a counted loop
4	CSE respects kills: redefined operand blocks reuse	33	selectHot covers the requested fraction
5	LICM hoists an invariant into a preheader	34	assignment of a binary expression is a single quad
6	LICM does not hoist a possibly-trapping division out of a zero-trip loop	35	mixed arithmetic inserts I2D conversion
7	LICM does not hoist a variant expression	36	short-circuit && does not evaluate its RHS
8	cold code is not transformed under profile direction	37	CFG blocks partition the function
9	RLE: a repeated load of the same element is removed	38	entry dominates every reachable block
10	RLE: a load right after a store reuses the stored value	39	nested natural loops are found, inner first
11	RLE: a store to the same array invalidates earlier loads, aliasing or not	40	liveness: loop-carried variable is live into the header
12	RLE: redefining the index invalidates the entry	41	IR interpreter matches expected semantics
13	RLE: a call invalidates global arrays but not local arrays	42	runtime errors are caught, not crashes
14	RLE: entries flow into a single-predecessor block but not across a join	43	minimal and full-featured programs are accepted
15	CSE then RLE removes the reloads of a compare-and-swap	44	operator precedence follows C++
16	DCE removes an unused computation and the chain feeding it	45	syntax errors
17	DCE keeps a value that is used	46	constructs outside the subset are rejected clearly
18	DCE never removes global writes or array stores	47	panic-mode recovery reports several syntax errors
19	DCE keeps a division that may trap	48	semantic errors
20	DCE removes a dead load	49	shadowing inner scopes gets unique IR names

#	Test case	#	Test case
21	DCE deletes unreachable code even when nothing is hot	50	narrowing double->int produces a warning, not an error
22	DCE leaves cold dead assignments alone	51	keywords, identifiers and punctuation
23	optimize() accepts rle and dce and rejects unknown names	52	maximal munch on multi-character operators
24	full pipeline with RLE and DCE preserves output and lowers energy	53	float literals with fraction and exponent
25	full pipeline preserves output and never increases energy	54	string escapes are unescaped
26	default cost table encodes memory >> ALU ordering	55	comments and preprocessor lines are skipped
27	quad categories	56	line and column tracking
28	cost file overrides entries	57	lexical errors are reported
29	exact trip count from constant bounds	58	multiple lexical errors are collected

Appendix D. Grammar of the Accepted Subset

```
program      = { top_decl } ;
top_decl     = "using" "namespace" "std" ";"
             | var_decl ";"
             | func_def ;
func_def     = type IDENT "(" [ params | "void" ] ")" block ;
params       = type IDENT { "," type IDENT } ;              (* scalars only *)
type         = "int" | "double" | "bool" | "void" ;
var_decl     = [ "const" ] type declarator { "," declarator } ;
declarator   = IDENT [ "[" expr "]" ] [ "=" ( expr | "{" [ expr { "," expr } ] "}" ) ] ;

block        = "{" { var_decl ";" | stmt } "}" ;
stmt         = block | ";"
             | "if" "(" expr ")" stmt [ "else" stmt ]
             | "while" "(" expr ")" stmt
             | "for" "(" [ for_init ] ";" [ expr ] ";" [ simple ] ")" stmt
             | "return" [ expr ] ";"
             | print ";"
             | simple ";" ;
for_init     = var_decl | simple ;                          (* one declarator *)
simple       = lvalue assign_op expr
             | lvalue ( "++" | "--" ) | ( "++" | "--" ) lvalue
             | call ;
assign_op    = "=" | "+=" | "-=" | "*=" | "/=" | "%=" ;
print        = [ "std" "::" ] "cout" "<<" item { "<<" item } ;
item         = STRING | CHAR | [ "std" "::" ] "endl" | additive ;

expr         = or_expr ;
or_expr      = and_expr { "||" and_expr } ;
and_expr     = equality { "&&" equality } ;
equality     = relational { ( "==" | "!=" ) relational } ;
relational   = additive { ( "<" | "<=" | ">" | ">=" ) additive } ;
additive     = multiplicative { ( "+" | "-" ) multiplicative } ;
multiplicative = unary { ( "*" | "/" | "%" ) unary } ;
unary        = ( "-" | "+" | "!" ) unary | postfix ;
postfix      = INT | FLOAT | "true" | "false" | "(" expr ")"
             | IDENT [ "(" [ expr { "," expr } ] ")" | "[" expr "]" ] ;
lvalue       = IDENT [ "[" expr "]" ] ;
```

Appendix E. Intermediate Representation

Quadruples `dst = a op b`, one per source-level operation. Names are x (local, shadowed locals become `x.1`), @g (global), `tN` (temporary) and `LN` (label). Every quad carries its source line.

Table 18. IR forms.

Form	Operations
`dst = a op b`	`Add Sub Mul Div Mod Shl Lt Le Gt Ge Eq Ne`
`dst = op a`	`Neg Not Copy I2D D2I`
`dst = a[b]` and `a[b] = c`	`Load` and `Store`

Form	Operations
`L:, goto L, iffalse a goto L`	`Label, Jmp, JmpF`
`param a, dst = call f, n, return a`	`Param, Call, Ret`
`print a, print "s", print endl`	`Print, PrintS, PrintNL`

Appendix F. Contribution Log (to be completed)

Table 19. Dated log of technical work. Add rows for each member.

Date	Member	Work item	Evidence (commit, PR, test)
date	name	work item	link
date	name	work item	link
date	name	work item	link
date	name	work item	link
date	name	work item	link
date	name	work item	link
date	name	work item	link
date	name	work item	link

Appendix G. Evaluator-Selected Test Record

To be filled in during the viva (Review 3 guidelines, section 8.1).

Table 20. Evaluator-selected tests.

Test / scenario	Expected behaviour	Observed output / evidence	Result / remarks
1. Evaluator-selected input			
2. Boundary or failure condition			
3. Rebuild / reproducibility check			